

## JDBC & Database Connectivity

### 1. What is JDBC?

JDBC (Java Database Connectivity) is an application programming interface (API) for the programming language Java, which defines how a client may access a database. It is a Java-based data access technology used for Java database connectivity.

It provides methods to query and update data in a database, and is oriented towards relational databases.

### 2. JDBC Architecture

The JDBC API supports both two-tier and three-tier processing models for database access but in general, JDBC Architecture consists of two layers:

- **JDBC API:** This provides the application-to-JDBC Manager connection.
- **JDBC Driver API:** This supports the JDBC Manager-to-Driver Connection.

### 3. Core JDBC Interfaces

- **DriverManager:** This class manages a list of database drivers. Matches connection requests from the java application with the proper database driver using communication sub protocol.
- **Connection:** Represents a session with a specific database. SQL statements are executed and results are returned within the context of a connection.

### 4. Statements and ResultSets

- **Statement:** Interface used to execute static SQL statements and return the results it produces.
- **PreparedStatement:** A precompiled SQL statement. This object can then be used to efficiently execute this statement multiple times. It also helps prevent SQL injection.
- **ResultSet:** Represents a database result set, which is usually generated by executing a statement that queries the database.

### 5. CRUD Operations Example Steps

To perform CRUD (Create, Read, Update, Delete) operations using JDBC:

- 1. Import JDBC packages.
- 2. Register JDBC driver (Class.forName).
- 3. Open a connection (DriverManager.getConnection).
- 4. Execute a query (using Statement or PreparedStatement).
- 5. Extract data from result set (for Read operations).
- 6. Clean up the environment (close ResultSet, Statement, Connection).